COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Oct 7, 2026	**DATE SUBMITTED:**	Oct 9, 2026
NAME:	Bal, Kevin Ranz, J. Braganza, Ralph Angelov F.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 11.1: Basic Algorithm Analysis

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
C/C++
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i]) != -1).{
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:

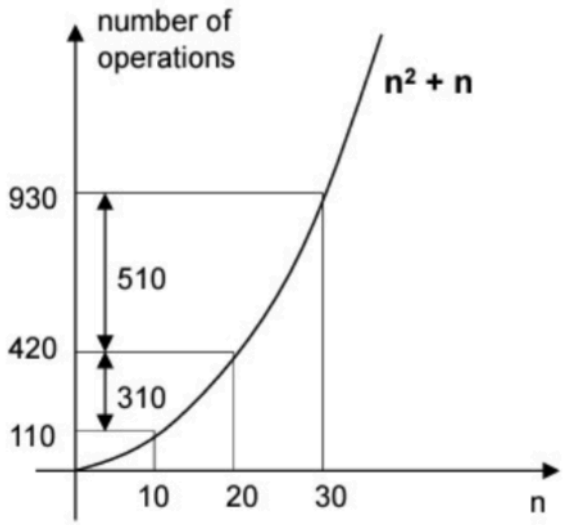
- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparison

Given the above information, the total number of comparisons in the worst case is:
O(mn)

- Assume that m = n. (The arrays are the same size

T(n)=n(n)
T(n) = n^2 (Frequency)
O(n^2) (Time Complexity)

Provide an analysis of the graph for the worst case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis

The first line defines a function that takes two integers as inputs and return either true or false at the end of the condition that both of them runs only **1 time** (x and y) while on the second line runs a for loop that inside it iterates it and execute it **only once** (i = 0;), and the second operation was the comparison of the (i) and (y.length) gives an **n+1**, and the third operation which is the incrementation runs **N times,** the fourth operation used an **if statement** takes the current item from the array y and runs a separate function to look for it inside the array x to check if it exist there and this line executes **N times**, while the return false; finds a match anywhere from the array x and if it does, it will stop the execution of the program in worse case and this **didn't run or 0 times.** Lastly, the return true; which checks every item in the array y and doesn't find any kind of matches to the array x, this will return or give true to confirm that the arrays are different and executes only once.

bool diff(int *x, int *y)	1
int i = 0;	1
i < y.length;	N+1
i++	N
if(search(x, y[i]) != -1)	N
return false;	Only run once but in worse case it's zero
return true;	1

Computation	1+1+1 = 3 N+1+N+N = 3N+1
Total	3 and 3N+1 or 3N+3

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Imagine that it is flu season and health department officials are planning to visit a school to ensure that all the enrolled children are administered their flu shot. However, there is a problem: a few children have already taken their flu shots but do not remember if they have been vaccinated against the specific category of flu that the health officials plan to vaccinate all the students against. Official records are sought out and the department is able to find a list of students that have already been administered the vaccine. A small excerpt of the list is shown here:

First Name	Last Name	Flu Shot?
1	3	Yes
2	2	No
2	3	Yes

Assume that all the names are positive integers and that the given list is sorted. Your task is to write a program that can look up the vaccination status of a given student in the list and outputs to the officials whether the student needs to be vaccinated. Students need to be vaccinated in case of two conditions:

- If they are not present in the list
- If they are present in the list but have not been administered a flu shot

Since the list can have a large number of students, your program should be as fast and efficient as possible. The final output of your program should look as follows:

```
Time taken to search = 45 microseconds
Student (836 118) needs vaccination.
Time taken to search = 37 microseconds
Student (836 118) needs vaccination.
Time taken to search = 53 microseconds
Student (836 118) needs vaccination.
```

The high-level steps for this implementation are as follows:

1. Represent each student as an object of the Student class.
2. Overload the required operators for the Student class so that a vector of students can be sorted using the STL `std::sort()` function.
3. Use a **binary search** to see if the student is present on the list.

4. If the student isn't present in the list, your function should return `true` since the student needs to be administered the vaccine.
5. Otherwise, if the student is present in the list but has not been administered the vaccine, return true.
6. Else, return false

Step 0: Begin by including the following headers:

```cpp
// Step 0: Include the required headers
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
```

Step 1: Define the STUDENT class

```cpp
// Step 1: Define the STUDENT class
class Student{
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    //constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
    }

    // Getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

```

Step 2: Overload the required operators for the Student class so that a vector of students can be sorted using the STL `std::sort()` function

```cpp
    // Step 2: Overload the required operators for the Student class

    // Two people are same if they have the same name
    bool operator ==(const Student& p) const {
        return this->name == p.name;
    }

    // The ordering of a set of people is defined by their name
    bool operator< (const Student& p) const {
        return this->name < p.name;
    }

    bool operator> (const Student& p) const {
        return this->name > p.name;
    }
};

```

Step 3: The following function lets us generate a student from random data:

```cpp
// Step 3: Generate a Student from random data
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;

    return Student(random_name, is_vaccinated);
}
```

For the next important function, we will implement the use of **std::chrono for experimental analysis**. The following code is used to run and test the output of our implementation.

```cpp
// Step 4: Implement experimental analysis using std::chrono
// For the next important function, we will implement the use of std::chrono for experimental analysis.
// The following code is used to run and test the output of our implementation.
void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
        std::chrono::steady_clock::now();

    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end =
        std::chrono::steady_clock::now();

    std::cout << "Time taken to search = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;

    if (search_result)
        std::cout << "Student (" << p.get_name().first << " "
        << p.get_name().second
        << ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " "
        << p.get_name().second
        << ") " << "does not need vaccination." << std::endl;
}
```

5. The following function implements our logic for whether a vaccination is needed:

```cpp
// Step 5: Implement the logic for determining whether vaccination is needed
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (true)
    {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);

        if (mid_element < P)
            std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}
```

Step 6: Implement the driver code

```
// Step 6: Implement the driver code
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);

    return 0;
}
```

BAL'S SYSTEM			
Input Size (N)	**Execution Speed**	**Screenshot**	**Observation(s)**
1000	1 microsecond/s.	`Time taken to search = 1 microseconds` `Student (534 410) needs vaccination.`	When running this code in the compiler, and given an input size of 1,000 the function inside the header checked to see if those students that were given inside the console needed a vaccination by splitting the array in half multiple times that ran a maximum of 10 loop operations. This splitting operation is useful because it lets the computer remove the 500 students at first operation and on the second one it gets splitted again from 500 to 250 instead of checking one of them each. Since the code rounds the timer down to a whole integer, the given output was time taken to search registered as 1 microsecond on the console with the help of the chrono library
10000	1 microsecond/s.	`Time taken to search = 1 microseconds` `Student (534 410) needs vaccination.`	When the input size was increased to 10,000, the program still ran the

			same on the first one. It first checks the 534 410 students for the vaccination from a large input size and starts again to divide it by two, where the 10,000 becomes 5,000,on the second operation it continues to repeat the division of two with a total of 14 total split operations to get the target. After that, it checks the condition and if it was confirmed that it equals to false and returns true for a vaccination since the students need it and stops the timer too. Adding 10,000 to the records of the test to the entire process and adding 4 more loop divisions compared to the first test that has a small size input of records that still gives a 1 microseconds of both programs but different input size.
100000	2 microseconds.	`Time taken to search = 2 microseconds` `Student (534 410) needs vaccination.`	When the input size was changed again from 10,000 to 100,000, program repeated the same process of the two example input size even though the size of this was big, it still determined the students that need vaccination that still same as the two example input size but their split operations are different, the first one was about 10 loop only, while the

			second one requires 14 split loop and this loop with a size of 100,000 requires a 17 total of split loop operation due to how big the input size is. It will start again from dividing it from two so it will become 50,000 and the second operation it will divide it to two to 17 times until it becomes smaller size based on the target and matching it to confirm if the students need a vaccination.

Table 11-1

BRAGANZA'S SYSTEM			
Input Size (N)	**Execution Speed**	**Screenshot**	**Observation(s)**

1000	0 microsecond/s.
	
	500 nanoseconds
		`Time taken to search = 0 microseconds`
		`Student (46 442) needs vaccination.`
		
		`Time taken to search = 500 nanoseconds`
		`Student (224 189) needs vaccination.`
			The search took 500 nanoseconds when the program executed needs_vaccination(p, people), which uses binary search to find the target student. Inside the function, range_length, mid_element_index, and mid_element are used to calculate and check the middle student, allowing the algorithm to eliminate part of the search range in each iteration.

10000	0 microsecond/s.
	
	600 nanoseconds
		`Time taken to search = 0 microseconds`
		`Student (46 442) needs vaccination.`
		
		`Time taken to search = 600 nanoseconds`
		`Student (224 189) needs vaccination.`
			The search took 600 nanoseconds, which was slightly longer than the first result. In the code, the conditions mid_element == P,

			mid_element > P, and mid_element < P determine whether the student has been found or whether the search should continue in a smaller range. If the target requires more iterations before reaching the matching student, the function may take longer to return a result.
100000	0 microseconds. 1000 nanoseconds	Time taken to search = 0 microseconds Student (46 442) needs vaccination. Time taken to search = 1000 nanoseconds Student (224 189) needs vaccination.	The search took 1,000 nanoseconds, the longest of the three measured runtimes. This test uses search_test(100000, p), which generates 100,000 students and sorts them before starting the timer; the sorting itself is not included in the recorded search time. The larger vector can require more binary-search iterations because the function repeatedly calculates the middle of the remaining range, although the actual runtime also depends on the target student and normal variations in computer execution.

Table 11-1

Analysis and Comparison (BAL)

The experimental results of My System and Braganza's System revealed that with an increase in the input size from 1,000 to 100,000 students, the two systems were able to provide relatively faster search times since binary search halves the search area rather than performing comparison against each individual student.

In contrast, Braganza's System recorded execution times of 500 nanoseconds, 600 nanoseconds, and 1,000

nanoseconds for input sizes of 1,000, 10,000, and 100,000 respectively, whereas My System recorded execution times of 1 microsecond for input sizes of 1,000 and 10,000 and 2 microseconds for 100,000. The possible cause of such a variance may be attributed to the processing capability of the systems, where Braganza's System utilizes RTX 5070 Ti along with Intel Core Ultra 7 255HX and my system uses Intel HD Graphics 630 and Intel Core i5-8400 processor it may not be the system I use at home but the difference is there .

Analysis and Comparison (BRAGANZA)

The experimental results of My System and Bal's System shows that as the input size increased from 1,000 to 100,000 students, both systems maintained relatively fast search times because binary search repeatedly divides the search range in half instead of checking every student individually.

On the other hand Bal's System recorded execution times of 1 microsecond for input sizes of 1,000 and 10,000, and 2 microseconds for 100,000. Meanwhile, my system recorded 500 nanoseconds, 600 nanoseconds, and 1,000 nanoseconds for the same input sizes, respectively (which is basically all 0 microseconds). This is because of the processing power of my system which has a RTX 5070 TI paired with a Ultra 7 255HX compared to Intel Graphics 630 (integrated) i5 8400 CPU.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis - big O notation / runtime / execution
- Experimental Analysis - code execution runtime / bytes and memory / behavior
- Analysis and Comparison - time and space complexity / linear or quadratic

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
    for i = 0 to n-1 do
     for j = i+1 to n-1 do
      if A[i] equals to A[j] then
      return false
    return true
```

CODE IMPLEMENTATION:

CPE010_HOA11p1_Braganza_sc3.cpp

```cpp
#include <iostream>
#include <vector>
#include <chrono>

// Algorithm Unique(A)
bool Unique(const std::vector<int>& A) {
    int n = A.size();

    // for i = 0 to n-1 do
    for (int i = 0; i < n; i++) {
        // for j = i+1 to n-1 do
        for (int j = i + 1; j < n; j++) {
            // if A[i] equals to A[j] then
            if (A[i] == A[j]) {
                // return false
                return false;
            }
        }
    }

    // return true
    return true;
}

int main() {
    int n = 5000;
    int repetitions = 100;

    // Best case: Duplicate values are found in the first comparison
    std::vector<int> bestCase(n, 1);
    bestCase[0] = 5;
    bestCase[1] = 5;

    // Worst case: All elements are unique
    std::vector<int> worstCase(n);
    for (int i = 0; i < n; i++) {
        worstCase[i] = i;
    }

    // Measure the best-case runtime
    auto startBest = std::chrono::steady_clock::now();
    bool bestResult;
    for (int i = 0; i < repetitions; i++) {
        bestResult = Unique(bestCase);
    }
    auto endBest = std::chrono::steady_clock::now();

    auto bestTime = std::chrono::duration_cast<std::chrono::microseconds>(endBest - startBest).count();

    // Measure the worst-case runtime
    auto startWorst = std::chrono::steady_clock::now();
    bool worstResult;
    for (int i = 0; i < repetitions; i++) {
        worstResult = Unique(worstCase);
    }
    auto endWorst = std::chrono::steady_clock::now();

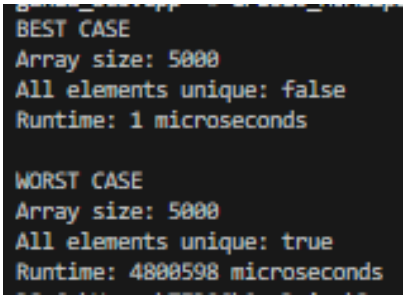
    auto worstTime = std::chrono::duration_cast<std::chrono::microseconds>(endWorst - startWorst).count();

    // Display the results
    std::cout << "BEST CASE" << std::endl;
    std::cout << "Array size: " << n << std::endl;
    std::cout << "All elements unique: " << std::boolalpha << bestResult << std::endl;
    std::cout << "Runtime: " << bestTime << " microseconds" << std::endl;

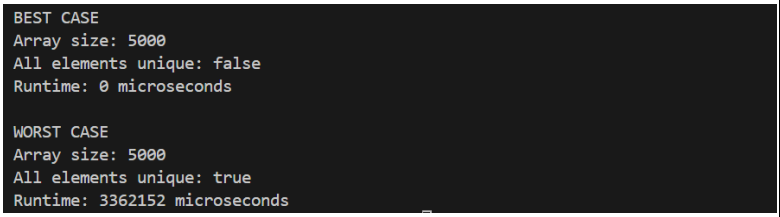
    std::cout << "\nWORST CASE" << std::endl;
    std::cout << "Array size: " << n << std::endl;
    std::cout << "All elements unique: " << std::boolalpha << worstResult << std::endl;
    std::cout << "Runtime: " << worstTime << " microseconds" << std::endl;

    return 0;
}
```

ILO A: Problem 1 (BAL)	
Theoretical Analysis	

	Best-Case | **O(1)** **Worst-Case** | **O(n^2)** This output evaluates the output of the algorithm by computing the number of operations that are performed based on the structure of each line of code. The program used nested loops to compare the array elements that are unique that are given on the function of the upper line of code of the program to find the duplicates of it. In the worst case part,where all of the elements are unique, the inner loop needs to execute for every element that gives a time complexity of O(n^2). While the best case is when the duplicate is found on the first comparison of it and stopping the loops immediately and giving a time complexity of O(1).
Experimental Analysis	 In this part, it measures the actual execution time of the program on an actual PC/Laptop using microseconds in STL chrono. It executes an array size of 5000, the best case found a duplicate and executes in 1 microseconds, while the worst case required to check every unique pair and took 4800598 microseconds to end the program

ILO A: Problem 1 (BRAGANZA)	
Theoretical Analysis	**Best-Case** | **O(1)** when a duplicate is found in the first comparison. **Worst-Case** | $\frac{n^2-1}{2}$ **or O(n^2)** when all elements are unique, which results in comparing each possible pair **A[i]=A[j]**. If a duplicate is found, the algorithm immediately returns false.

	The algorithm has a **worst-case time complexity of O(n^2)** because it compares every possible pair of elements when no duplicates exist. As the input size increases, the number of comparisons grows quadratically. The **best-case** occurs when a duplicate is found in the very first comparison, requiring only one comparison, **O(n).**
Experimental Analysis	 In the **best case**, the algorithm returned false in **0 microseconds** because **it found duplicate values in the first comparison** and immediately stopped checking the remaining elements. In the **worst case**, the algorithm returned true in **3,362,152 microseconds**, because all 5,000 elements were unique. As a result, the algorithm had to **compare every pair of elements before confirming that there were no duplicates.**

Analysis and Comparison (BAL)

The comparison of both data demonstrates a clear and direct connection between the test measurements and the theoretical time complexities of it for each case. For the best case, the runtime on it was quick at an output of **1 microsecond** because the **theoretical O(1)** allowed the program to see the duplicates and exit immediately **without looping the 5000 size of it**. For the worst case, the execution jump to **4800598 microsecond** because of the time **complexity of O(n^2)** to do too much comparison because it had to **check the each combination before confirming that 5000 size was unique**

Analysis and Comparison (BRAGANZA)

The experimental results of my system and Bal's system show that both algorithms perform differently depending on whether the array contains duplicate values. In the best case, both systems stop early after finding a duplicate, resulting in very short execution times. My System recorded **0 microseconds**, while Bal's System recorded **1 microsecond**. In the worst case, both systems had to compare every possible pair of elements because **all 5,000 elements were unique**. My system took **3,362,152 microseconds**, while Bal's System took **4,800,598 microseconds**. This means My system was faster by **1,438,446 microseconds (I had to actually compute this)** in the worst case. Overall, both systems **demonstrate the same theoretical time complexity of O(n^2)** in the worst case because the number of comparisons increases quadratically as the input size grows.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
    if n == 0 return 1
    else return x*rpower(x, n-1)
```

```
Algorithm brpower(int x, int n):
 if n == 0 return 1
 if n is odd then
     y = brpower(x, (n-1)/2)
     return x*y*y
 if n is even then
     y = brpower(x, n/2)
     return y*y
```

CODE IMPLEMENTATION:

CPE010_HOA11p1_Braganza_sc4.cpp

```cpp
#include <iostream>
#include <chrono>

// Algorithm rpower(int x, int n)
int rpower(int x, int n) {
    // if n == 0 return 1
    if (n == 0)
        return 1;
    // else return x * rpower(x, n-1)
    else
        return x * rpower(x, n - 1);
}

// Algorithm brpower(int x, int n)
int brpower(int x, int n) {
    // if n == 0 return 1
    if (n == 0)
        return 1;
    // if n is odd
    if (n % 2 != 0) {
        // y = brpower(x, (n-1)/2)
        int y = brpower(x, (n - 1) / 2);
        // return x * y * y
        return x * y * y;
    }
    // if n is even
    if (n % 2 == 0) {
        // y = brpower(x, n/2)
        int y = brpower(x, n / 2);
        // return y * y
        return y * y;
    }
    return 1;
}

int main() {
    int x = 2;
    int bestN = 2;
    int worstN = 25;
    int repetitions = 1000000;

    int rBestResult, rWorstResult;
    int brBestResult, brWorstResult;

    // Measure rpower best-case runtime
    auto startRBest = std::chrono::steady_clock::now();
    for (int i = 0; i < repetitions; i++) {
```

```cpp
    for (int i = 0; i < repetitions; i++) {
        rBestResult = rpower(x, bestN);
    }
    auto endRBest = std::chrono::steady_clock::now();
    auto rBestTime = std::chrono::duration_cast<std::chrono::microseconds>(endRBest - startRBest).count();

    // Measure rpower worst-case runtime
    auto startRWorst = std::chrono::steady_clock::now();
    for (int i = 0; i < repetitions; i++) {
        rWorstResult = rpower(x, worstN);
    }
    auto endRWorst = std::chrono::steady_clock::now();
    auto rWorstTime = std::chrono::duration_cast<std::chrono::microseconds>(endRWorst - startRWorst).count();

    // Measure brpower best-case runtime
    auto startBRBest = std::chrono::steady_clock::now();
    for (int i = 0; i < repetitions; i++) {
        brBestResult = brpower(x, bestN);
    }
    auto endBRBest = std::chrono::steady_clock::now();
    auto brBestTime = std::chrono::duration_cast<std::chrono::microseconds>(endBRBest - startBRBest).count();

    // Measure brpower worst-case runtime
    auto startBRWorst = std::chrono::steady_clock::now();
    for (int i = 0; i < repetitions; i++) {
        brWorstResult = brpower(x, worstN);
    }
    auto endBRWorst = std::chrono::steady_clock::now();
    auto brWorstTime = std::chrono::duration_cast<std::chrono::microseconds>(endBRWorst - startBRWorst).count();

    // Display rpower results
    std::cout << "| RPOWER |" << std::endl;
    std::cout << "\nBEST CASE" << std::endl;
    std::cout << "Exponent: " << bestN << std::endl;
    std::cout << "Result: " << rBestResult << std::endl;
    std::cout << "Runtime: " << rBestTime << " microseconds" << std::endl;

    std::cout << "\nWORST CASE" << std::endl;
    std::cout << "Exponent: " << worstN << std::endl;
    std::cout << "Result: " << rWorstResult << std::endl;
    std::cout << "Runtime: " << rWorstTime << " microseconds" << std::endl;

    // Display brpower results
    std::cout << "\n| BRPOWER |" << std::endl;

    std::cout << "\nBEST CASE" << std::endl;
    std::cout << "Exponent: " << bestN << std::endl;
    std::cout << "Result: " << brBestResult << std::endl;
    std::cout << "Runtime: " << brBestTime << " microseconds" << std::endl;

    std::cout << "\nWORST CASE" << std::endl;
    std::cout << "Exponent: " << worstN << std::endl;
    std::cout << "Result: " << brWorstResult << std::endl;
    std::cout << "Runtime: " << brWorstTime << " microseconds" << std::endl;

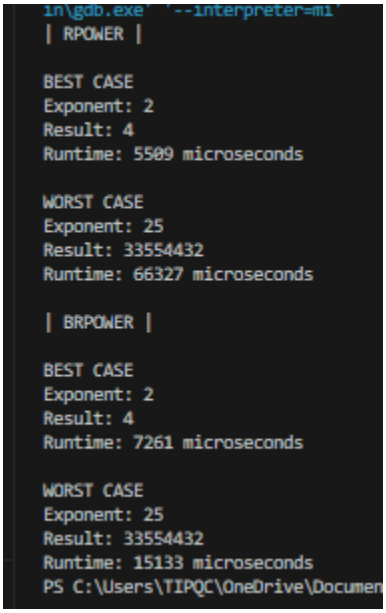
    return 0;
}
```

ILO A: Problem 2 (BAL)

Theoretical Analysis

rpower(x, n)	
Best-Case	**O(1)**
Worst-Case	**O(n)**

The operations for rpower(x,n) decreases the exponent by 1 each repeating step, means, it must perform a long order of steps to reach the best case which results in a linear time complexity of **O(n)** for both best and worst case. because the countdown step always stays the same.

	<table><tr><th colspan="2">brpower(x, n)</th></tr><tr><td>Best-Case</td><td>O(1)</td></tr><tr><td>Worst-Case</td><td>O(log n)</td></tr></table> In the brpower(x, n), it applies a binary division that will cut the exponent value in half during each step, instead of subtracting it to 1, allowing it to skip a lot of individual multiplications by using squares. This method reduces the total number of operations needed, giving it a time complexity of **O(log n)** for both its best and worst case because dividing the number down makes the function less work and quick no matter the of the starting of the exponent size
Experimental Analysis	 For the rpower, the best case of its runtime was **5,509 microseconds** with an **exponent of 2** and the worst case runtime of it was 66,327 microseconds with an **exponent of 25**. For the brpower, the best case of its runtime was **7,261 microseconds** with an exponent of 2, while the worst case of it has a **15,133 microseconds with an exponent of 25**

ILO A: Problem 2 (BRAGANZA)

Theoretical Analysis

rpower(x, n)	
Best-Case	**O(1)** when **n=0**. The algorithm immediately checks the condition if n == 0 and returns 1.
Worst-Case	**O(n)** when **n is greater than 0**. The algorithm repeatedly **subtracts 1 from n until it reaches 0.**

brpower(x, n)	
Best-Case	**O(1)** when **n=0,** the algorithm immediately returns 1 without making any recursive calls.
Worst-Case	**O(log n)** when n **is greater than 0** BUT instead of subtracting 1 each time, the algorithm **divides n by 2** during every recursive call. For example, if n=16, the exponent decreases as 16 ÷ 2 = 8 8 ÷ 2 = 4 4 ÷ 2 = 2 2 ÷ 2 = 1 1 ÷ 2 = 0 **Both algorithms have a best-case time complexity of O(n) when n = 0**, because they immediately return 1 without making recursive calls. However, their worst-case time complexities are different because the **rpower** algorithm has a time complexity of **O(n) because it subtracts 1** from the exponent in every recursive call, while **brpower has a time complexity of O(log n) because it divides the exponent by 2 each time.**

Experimental Analysis	 For **rpower(), the best-case** test used an exponent of 2 and produced a **result of 4 with a runtime of 2,406 microseconds.** When the exponent was **increased to 25**, the result was **33,554,432**, and the runtime **increased to 24,158 microseconds.** This is because rpower() decreases the exponent by one during each recursive call, **requiring more recursive calls as the exponent increases.** On the other hand, **brpower()**, the best-case test also used an exponent of 2 and produced a result of 4 with a **runtime of 2,288 microseconds.** With an exponent of 25, the **result was 33,554,432**, and the **runtime increased to 9,293 microseconds**. The experimental results show that brpower() was faster than rpower() for the exponent of 25. This supports the **theoretical analysis that rpower() has a time complexity of O(n), while brpower() has a time complexity of O(logn).**

Analysis and Comparison (BAL)

This demonstrates a clear link between the test measurements and the theoretical time complexities for each case. For the **rpower algorithm**, the physical runtime grew from **5,509 microseconds** in the best case to 66,327 microseconds in the worst case because the linear complexity requires many more steps as the exponent gets larger. For the brpower algorithm, the numbers show a flatter growth curve where the best case took **7,261 microseconds** and the worst case took **15,133 microseconds**, proving how much better the **O(logn) controls runtime since multiplying the input size by twelve did not even double the execution time**. When comparing the two methods directly, **rpower actually performed a little better in the best case due to less internal conditional code setup**, but brpower completely finished faster in the worst case by finishing four times faster because it completely bypasses the pile of individual multiplications.

Analysis and Comparison (BRAGANZA)

The experimental results of My system and Bal's system show that both rpower() and brpower() produce the same results but have different execution times. For an **exponent of 2**, both algorithms completed the calculation in a relatively short time. However, when the exponent increased to 25, the execution time

increased for both algorithms. In my system, rpower() took **24,158 microseconds**, while brpower() took only **9,293 microseconds**. Similarly, in Bal's system, rpower() took **66,327 microseconds,** while brpower() took **15,133 microseconds**. These results show that brpower() was faster than rpower() when calculating a larger exponent in both systems. This supports the theoretical analysis that rpower() has a time complexity of O(n) because it decreases the exponent by one in every recursive call, while brpower() has a time complexity of O(log n) because it divides the exponent by two in each recursive call but even when O(n) is relatively faster than O(log n) it handles larger data better.

CONCLUSION (BRAGANZA)

In this hands-on activity, I found out how binary search operates and how it is able to find the desired student efficiently. Binary search continuously reduces the range of search by half which makes it have O(logn) time complexity (one of the fastest time complexities). In addition, I found out how to use the chrono library in C++ to compute execution times in microseconds and nanoseconds. Moreover, I found out how computer hardware and input size affect execution times. I feel really happy that I managed to work with the chrono library because I could do many interesting things with such functions that it provides in its library. in C++.

The process helped me learn how to apply binary search by using C++ and also how to calculate its time of execution by using std::chrono. The program created random information about students, sorted students by their names and then found certain students to check if vaccination is necessary. As the number of input values grows from 1,000 up to 100,000, the time required for the search increases only slightly because the algorithm does not have to compare every single student. This task taught me the main value of algorithm efficiency.

The supplementary activity allowed me to compare my system, Ba'ls System, with Bal's System by testing both programs using different input sizes. My System recorded search times of 500, 600, and 1,000 nanoseconds, while Bal's System recorded 1, 1, and 2 microseconds for input sizes of 1,000, 10,000, and 100,000. Although my system recorded lower execution times, these differences may be influenced by my hardware specifications which gives me the realization that I really have to optimize my code if I ever want to develop a project that others will use.

I think that I did well in this task as I successfully implemented the program, measured execution time, and compared the results with another system. But at the same time, I have to admit that I should really improve my knowledge about the working principle of time complexity and space complexity. I also should conduct more results/outputs by measuring the average execution time for several trials because our initial program didn't have that much significant difference so we had to increase the array size to showcase it. Overall, this activity helped me understand the importance of algorithm efficiency and how experimental results can be used to evaluate program performance.

CONCLUSION (BAL)

In this activity, this activity demonstrated and display those results of each examples mostly on supplementary part, like the exact result output, actual runtimes on a PC/Laptop by testing different kinds of programs like finding duplicate in a list and calculating an exponents and displayed a clear layout of the program and controls

the execution speed of nanoseconds and microseconds when handling a big amount sets of data. Showing an executed console that finished immediately when finding an early match of it, while the other one slows down completely because it had to check every single combination made the time complexity clear to understand.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Google Overview (Gemini) and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, idea generation, formatting, summarization, code assistance, and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It was used for code assistance to debut problems with the chrono timing and logic but most of it was referenced from the procedure. It was also used for for grammar correction and better explanation.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.